Specification of the clautolisp Debugger

Draft v0.1

June 24, 2026

Contents

- **Status:** Draft v0.1
- **Scope:** Debugger, inspector, and three pluggable user interfaces for clautolisp — an AutoLISP implementation hosted on Common Lisp.
- **Audience:** Implementers of clautolisp and of debugger user interfaces against it.

Foreword and credits

The architectural backbone of this specification — same-process two-thread debugger, two function bodies per function selected by entry point, poll points with a Bloom-filter fast path, stepping as volatile breakpoints, command-queue protocol with no wire format — is taken directly from Robert Strandh's work on the SICL debugger and its prototype Clordane. Where his design applies as-is, we adopt it; where AutoLISP semantics or clautolisp's hosting on Common Lisp require adaptation, we say so explicitly.

Key references (BibTeX at end of document):

- Strandh, *Omnipresent and low-overhead application debugging*, ELS 2020 [Strandh2020]. The design document for the SICL debugger.
- Strandh, *Clordane: Debugger for Common Lisp systems*, `https://github.com/robert-strandh/` [Clordane]. Prototype implementation and the user-facing manual (`Documentation/chap-*.tex`) whose vocabulary and stepping semantics we lift wholesale.

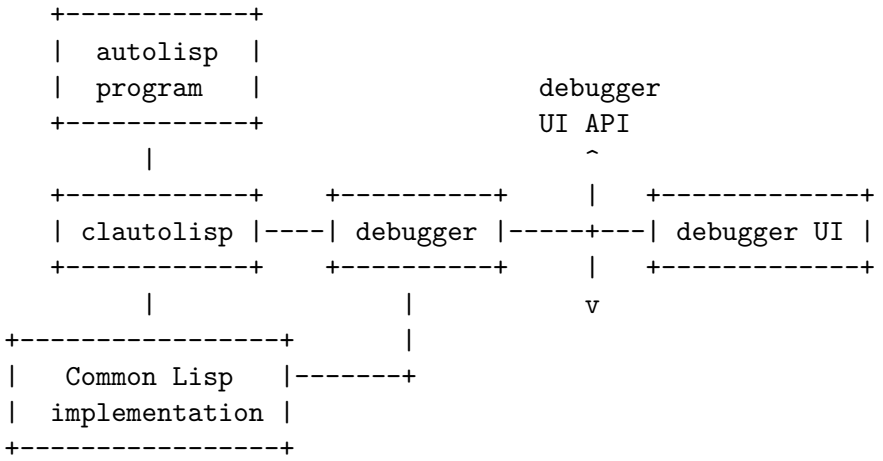
- Frode Fjeld is credited in [Strandh2020] for the multiple-entry-points-per-function idea; Michael Raskin is credited for the two-function-bodies idea. We follow both.

The AutoLISP error-handling primitives (the `*error*` function-valued global, and `vl-catch-all-apply`/`=vl-catch-all-error-p`/`=vl-catch-all-error-message`) are documented in Autodesk's *AutoLISP Developer's Guide* [AutoLISP-DG].

Part I — Foundations

1. Overview and architecture

1.1 Position in the system



Three layers, three concerns:

- **clautolisp** runs the AutoLISP program. It is responsible for *cooperation* with the debugger via the `clautolisp.debug` protocol (Part III).
- **debugger** is a Common Lisp library implementing the debugger logic — breakpoint management, stepping, the inspector, the snapshot of program state at a stopping point. It is UI-independent.
- **debugger UI** is one of three interchangeable front-ends (dumb terminal, ncurses, Emacs) implementing the `debugger.ui` protocol (Part V).

The debugger and the AutoLISP program run **in the same Common Lisp image, in different threads**, following [Strandh2020 §4]. There is no wire protocol between debugger and application; communication is via shared Lisp data plus a `bordeaux-threads` queue and semaphore.

The debugger UI may run in the same image (terminal UIs) or in a separate process (Emacs UI), connected to the debugger by a thin RPC. The UI/debugger boundary is the only place a wire protocol may appear, and only when the chosen UI requires it.

1.2 Threads

We adopt Strandh's terminology [Clordane Documentation/chap-intro.tex]:

- An **application thread** is a thread running AutoLISP code under debugger control. There may be more than one.
- The **debugger thread** is the thread running the debugger's command loop. Exactly one per session.
- A **UI thread** is a thread running a particular debugger UI. For in-image UIs this is typically the thread that drove the session start; for the Emacs UI it is a worker thread that handles RPC.

A debugger thread cannot debug itself. It can debug any other thread, including other instances of itself running in a different thread.

1.3 Why not just use the Common Lisp debugger?

clautolisp is implemented in CL and could in principle defer to the host's debugger (SBCL's debugger, SLDB, etc.). We do not, for three reasons:

1. The AutoLISP programmer should see AutoLISP source, AutoLISP values, AutoLISP call frames — not the host's. A stack frame from inside `EVAL-AUTOLISP-FORM` is noise.
2. AutoLISP has no condition system. The host debugger's affordances (restarts, `HANDLER-CASE`, `INVOKE-DEBUGGER`) have no direct AutoLISP analog and would confuse the user.
3. Source-level stepping requires cooperation from the *interpreter or compiler of AutoLISP*, not from the host implementation.

The host's debugger remains available to clautolisp *implementers* (the system programmer in Strandh's sense [Strandh2020 §3]) when debugging clautolisp itself. The clautolisp debugger specified here targets only the application programmer.

2. Terminology

Adopted from [Clordane Documentation/chap-intro.tex], adapted where needed:

- **Poll point.** A position in compiled or interpreted AutoLISP code where execution can be safely stopped. The interpreter or compiler designates poll points; they correspond to source-level positions (typically the beginning and end of each form). See §11.
- **Breakpoint.** A poll point that has been marked active, so that the application thread will stop when control reaches it.
  - A breakpoint is **steady** if it survives breakpoint hits, or **volatile** if it is removed the first time *any* breakpoint is hit on the relevant thread. Stepping is implemented entirely with volatile breakpoints.
  - A breakpoint may have an associated *condition* — a CL form to evaluate in the debugger thread, with access to the application's live variables and to debugger workspace slots (§16.2); the breakpoint fires only if the form returns non-`NIL`.
  - A breakpoint may have an associated *trace action* — a form whose evaluation produces a trace message and which transparently continues the application (see §6.4).
- **Stopping point.** The current execution position of an application thread when it is stopped. Always a poll point under normal control flow; can be at an arbitrary instruction only after instruction-level stepping (§6.3.2) — clautolisp's interpreter has no notion of "instruction", so this case is absent from the v1 spec.
- **Source position.** A `(start, end)` pair, where each endpoint is a `(file, line, column)` triple. Produced by the reader (§3) and carried on every form.
- **Form id.** A small integer assigned by the compiler/interpreter to each form within a function, used as a poll-point identity. See §11.

- **Continue.** Resume execution of the stopped application thread until termination or until another breakpoint is hit.
- **Advance.** Set a volatile breakpoint at a user-chosen poll point and continue. The volatile breakpoint is removed when hit.
- **Step in / out / over / call.** Move execution to a structurally-defined next poll point. Defined in §6.3.

3. Required support from the reader

The clautolisp reader already tracks source positions of symbols and tokens. The debugger requires:

1. Every cons cell produced by reading a list inherits a source position covering the open paren through the close paren.
2. Every atom retains its source position.
3. Source positions survive macro expansion; an expanded form's source position is inherited from the form that introduced it, with provenance recorded so the inspector can show the unexpanded form.
4. Source positions are accessible through an API:

```
(clautolisp.source:position-of object) → source-position | NIL
(clautolisp.source:lines-of file)      → simple-vector of strings
```

5. The `source-position` object is `EQUAL`-comparable and `PRINT`-able.

This is the same shape of API as `sicl-source-tracking` used in [Clordane] (see `Clordane/Code/gui.lisp` lines 23–37 for an example of consumption).

4. Required support from the runtime

clautolisp must expose the following to the debugger library, in a single package `clautolisp.debug`:

- A *debug flag* per application thread (initially clear). When clear, all instrumentation is a no-op modulo the cost of one load and one branch.

- A *breakpoint table* per debugged thread, mapping form-ids to breakpoint records.
- A *summary bit table* per debugged thread, sized 1024 bits, acting as a Bloom filter over the breakpoint table — see §11.2 and [Strandh2020 §4].
- A *communication queue* per debugged thread (`bordeaux-threads` blocking queue with associated semaphore).
- The currently-executing function and form id (the *current poll-point*), maintained as part of the instrumentation contract.
- A snapshot procedure that captures, on demand, the live AutoLISP environment of the application thread: the chain of bindings visible at the current poll point, plus the AutoLISP call stack.

This contract is specified in detail in Part III.

Part II — Compilation and instrumentation

5. Two function bodies

Following [Strandh2020 §4] (credited there to Michael Raskin), each AutoLISP function compiled by clautolisp produces **two CL function bodies**:

- The **ordinary body** is fully optimized (CL declarations as the clautolisp compiler sees fit, no instrumentation).
- The **debugging body** has most optimizations disabled and contains explicit poll-point calls before and after every source-level form.

The two bodies share the same name and arity; selection is by *entry point* (Fjeld's mechanism, [Strandh2020 §4]):

- A function-call site in *ordinary* code calls the ordinary entry point of the callee.
- A function-call site in *debugging* code calls the debugging entry point of the callee.

Debugging-ness propagates automatically down the call chain: once you enter a function through its debugging entry point, every callee is reached through *its* debugging entry point. This is what makes the system *omnipresent* in Strandh's sense — the user never has to decide what to compile for debugging and what for production.

5.1 Concrete encoding in CL

For an AutoLISP definition `(defun FOO (a b) ...)`, the clautolisp compiler emits:

```
(defun foo/ordinary (a b)
  <ordinary body - optimised>)

(defun foo/debug (a b)
  <debugging body - instrumented>)

(setf (clautolisp.runtime:function-entries 'foo)
      (vector #'foo/ordinary #'foo/debug))
```

A call site in *ordinary* code becomes a call to `foo/ordinary` (resolved at clautolisp-compile time via the entries vector, or directly inlined where possible).

A call site in *debugging* code becomes a call to `foo/debug`.

For interpreted code (no compiler yet, see §13), the interpreter has an explicit `*debug-mode*` flag and dispatches the two body shapes from the same AST.

5.2 Optimisations disabled in the debugging body

Following [Strandh2020 §4]:

- No common sub-expression elimination across poll points.
- No dead-code elimination of in-scope-but-unused intermediate values that correspond to a sub-form's result; each such result is kept available until the form's `:after` poll point. (This does *not* apply to the AutoLISP-level binding stack — AutoLISP variables are dynamic and live in the runtime's binding stack regardless of compiler choices; see §9.)
- No loop-invariant code motion across poll points.

- No tail-call optimisation in the debugging body (the user wants to see the call chain).
- No type-inference propagation across poll points (this is what makes safe variable assignment through the debugger possible — see §9.5).

Inlining is permitted within a single source form (the inlined body's poll points are absorbed into the caller's form), but not across functions; the debugging entry point of a callee is always called.

5.3 Instrumentation: what the debugging body actually contains

For each AutoLISP form `F` with form-id k, the debugging body wraps the CL-translation of `F` as:

```
(progn
  (clautolisp.debug:poll-point k :before)
  (multiple-value-prog1 <translated-F>
    (clautolisp.debug:poll-point k :after)))
```

`poll-point` is the routine described in §11.

For special forms with sub-forms (`if`, `cond`, `progn`, `setq`, `while`, `repeat`, `foreach`, `lambda` bodies), each sub-form is itself wrapped. The form-id assignment is dense and contiguous within a function; the compiler produces, as a side effect, a *form-id table* mapping `k` → source position, captured in the function's debug metadata (§7).

For `defun`, the body's outer `progn` is wrapped with a *function-entry* poll point and a *function-exit* poll point; these are distinguishable from inner poll points by tag, so "step out" / "step call" can find them in O(1).

6. Stepping

Stepping is implemented entirely as **volatile breakpoints inserted by the debugger** before continuing the application thread [Strandh2020 §4, slide 21–22]. The runtime has no notion of stepping; it only knows about breakpoints.

Once *any* breakpoint (steady or volatile) fires, all volatile breakpoints on the relevant thread are cleared. This is the only mechanism by which volatile breakpoints are removed [Clordane chap-intro.tex].

6.1 Forms and positions

The stepping vocabulary uses **at the beginning of form F** and **at the end of form F** as primitive positions [Clordane chap-stepping.tex]. Each maps to a poll point.

The compiler emits two poll points per form: one *before* (the beginning) and one *after* (the end). Both are recorded in the form-id table.

6.2 Stepping commands — semantics

We adopt the precise case-by-case semantics from [Clordane Documentation/chap-stepping.tex] verbatim. The reader is referred there for the full definitions; we summarise the volatile-breakpoint placement strategy here.

Step over. Place a volatile breakpoint at the end of the form immediately following the current stopping point in evaluation order. If the immediately enclosing form is fully evaluated, place it at the end of the enclosing form (this case recurses).

Step in. Place a volatile breakpoint at the beginning of the first sub-form that needs evaluation. If the current stopping point is at the end of the last sub-form and the enclosing form is a function call, place it at the function-entry poll point of the called function.

Step out. Place a volatile breakpoint at the end of the immediately enclosing form (or, if the enclosing form is the function body, at the function-exit poll point, with subsequent stop at the corresponding "after" poll point of the call site — see [Clordane chap-stepping.tex §step-out]).

Step call. Only meaningful at the beginning of a function-call form. Place a volatile breakpoint at the function-entry poll point of the callee. Distinguished from step in because step in would stop *before* argument evaluation; step call stops *after* arguments are evaluated and inside the callee.

Finish. Alias for step out, where the enclosing form is taken to be the entire function body.

Advance to point. User clicks (or otherwise indicates) a poll point in a source view. The debugger places a volatile breakpoint there and continues.

6.3 Non-local exits

If the form evaluated during stepping performs a non-local exit (AutoLISP has no `THROW` directly but does have early function return through interpreter mechanisms, plus `vl-catch-all-apply`-mediated unwinding), execution will stop only if *some* breakpoint is reached on the way out [Clordane chap-stepping.tex]. The debugger guarantees nothing about stopping at a

specific form in that case; it does guarantee that all volatile breakpoints are cleared at the first hit, so the program won't accumulate spurious stops.

6.4 Tracing

Tracing is a tagged breakpoint [Strandh2020 slide 22]. A trace point has an associated *trace form* (a CL form, evaluated by the debugger in an environment that exposes the application's live variables — see §9.2). When the application hits a trace point:

1. The application enqueues a *hit* notification on its outbound queue and blocks on its inbound queue.
2. The debugger receives the hit, looks up the breakpoint record, sees it is a trace point, evaluates the trace form (whose typical body is `(format ...)` to a trace stream), and immediately enqueues a *continue* message.
3. The application is unblocked and resumes.

This is morally identical to a conditional breakpoint whose condition is `(progn <print> nil)` plus auto-continue; we keep it as a distinct kind for clarity and because the UI presents it separately. Crucially, tracing does **not** modify the application code, and does **not** require encapsulation [Strandh2020 §4, §5.2]. There is therefore no restriction on which functions can be traced — including functions used internally by `print` or `format`-equivalents, because the debugger thread does the printing.

7. Debug metadata

Each compiled AutoLISP function carries a *debug metadata record* attached to it (in CL, an entry in a weak hash table keyed by the function object, or a slot on the function's structure if functions are CLOS-backed):

```
(defstruct function-debug-metadata
  source-position          ; of the defun form
  form-id->position        ; vector indexed by form-id → source-position
  form-id->kind            ; vector → :before-form | :after-form |
                           ;          :function-entry | :function-exit
  poll-point-count         ; integer
  bound-names              ; list of AutoLISP symbols dynamically bound by
                           ;   this function - i.e. its formals and the
```

```
                          ;   /-locals; see §9
  parent-form-map         ; vector: form-id → enclosing form-id | NIL
  source-text             ; reference to source-text buffer / file
  )
```

`bound-names` is the list of AutoLISP symbols this function shadows on entry: its formal parameters and the locals declared after `/` in the lambda list. The list is in *binding order*, so that the snapshot machinery (§9) can reconstruct which name was pushed when. The names themselves are AutoLISP symbols (uppercase per convention) and have no per-function place descriptor — values live in the runtime's binding stack, not in compiler-allocated slots (§9.2).

For interpreted code (§13), the same record is built at parse time and attached to the AutoLISP function object (which is itself a structure or CLOS instance in clautolisp).

Part III — The debug protocol (clautolisp debugger)

The package `clautolisp.debug` exports the contract between the runtime and the debugger. **Both sides** of this contract live in the same image, in different threads.

8. Communication channel

8.1 Per-thread state

Each *debugged* application thread has an associated `thread-debug-info`:

```
(defclass thread-debug-info ()
  ((thread             :reader  ti-thread)
   (debug-flag         :accessor ti-debug-flag      :initform nil)
   (breakpoint-table   :reader  ti-breakpoints
                       :initform (make-breakpoint-table))
   (summary-bits       :reader  ti-summary
                       :initform (make-array 1024 :element-type 'bit
                                                  :initial-element 0))
   (inbound-queue      :reader  ti-inbound)   ; debugger → app
   (outbound-queue     :reader  ti-outbound)  ; app → debugger
   (current-pp         :accessor ti-current-pp :initform nil)
   (status             :accessor ti-status :initform :running)))
```

The breakpoint table is logically keyed by `(function, form-id)`. Implementations choose between a two-level table (outer keyed by `function`, inner an `fixnum`-keyed table or vector indexed by `form-id`), or a single table with a precomputed `fixnum` hash combining the two. What matters is that the hot path of §11.1 incurs no allocation. The summary-bits index uses the same combining hash, taken modulo 1024, following the Bloom-filter discipline of [Strandh2020 §4].

The two queues are `bordeaux-threads` blocking queues (or the equivalent from `chanl` / `lparallel`); the precise implementation is unimportant as long as `enqueue` is non-blocking and `dequeue` blocks on empty.

A central registry `*thread-debug-info-table*` maps `bt:thread` → `thread-debug-info` for all currently debugged threads.

8.2 Message kinds, app → debugger

```
:hit             form-id, function, kind (steady/volatile/trace), live snapshot
:unhandled-error  AutoLISP error string (from *error*-equivalent path)
:caught-error     vl-catch-all error object (see §10)
:thread-exit     value-or-error
```

8.3 Message kinds, debugger → app

```
:continue
:set-variable   binding-entry, new-value (validated, see §9.5)
:eval-result    result of an eval-in-frame request (the request is satisfied
                synchronously from a debugger callback; this kind is only for
                async eval, deferred from v1)
```

The protocol is request/response on a single connection. After a `:hit` the application is blocked on its inbound queue until the debugger enqueues a `:continue`. Between those two events, the debugger may inspect the snapshot freely, evaluate forms in the snapshot environment (synchronously, since it has direct CL access — no message needed), and add/remove breakpoints by writing the breakpoint-table directly.

9. Snapshot and the live environment

9.1 AutoLISP variable model

AutoLISP has **no lexical binding**. Every variable name resolves through a single dynamic mechanism: at any point in execution, the value of a symbol

is its current dynamic binding. There are two flavours of binding by *origin*, not by scoping rule:

- **Global bindings.** Set by `(setq foo ...)` at top level or by a `setq` that names a symbol with no current shadowing. These persist until reassigned or the AutoLISP image ends.
- **Frame-local bindings.** A `defun`'s formal parameters and the names declared after `/` in its lambda list are *dynamically shadowed* on function entry: their previous binding (whatever it was — global, or another frame's shadow) is saved, a fresh binding is established, and on function exit the previous binding is restored. This is exactly the semantics of CL's `defvar`-style dynamic variables, scoped to the dynamic extent of the function call.

A consequence: a function deep in the call stack sees the *innermost* binding of a name, regardless of which frame established it. Two frames in the call stack may have introduced shadowings for the same name; only the innermost is "live" by ordinary lookup, but the outer shadows are still on the binding stack and the debugger can reach them.

9.2 What a snapshot contains

At a `:hit`, the application produces a snapshot:

```
(defstruct snapshot
  thread                    ; the application thread
  function                  ; the AutoLISP function executing
  form-id                   ; current poll point
  source-position           ; resolved from form-id via debug metadata
  call-stack                ; list of frames, innermost first; see §9.3
  binding-stack             ; list of bindings, innermost first; see §9.4
  visible-names             ; alist: AutoLISP symbol → innermost value
                            ;        (the result of ordinary lookup at
                            ;        the stopping point)
  globals-touched           ; alist: AutoLISP symbol → value, for global
                            ;        names that have been set or referenced
                            ;        in this thread since session start
                            ;        (see §9.5)
  catch-stack               ; list of active vl-catch-all-apply frames
                            ; (see §10)
  )
```

`visible-names` is what the AutoLISP programmer expects when they type a variable name into the debugger REPL — the value they would get if they evaluated that symbol from a form executed at this stopping point. It is computed by walking the binding stack innermost-first and taking the first binding for each name.

`binding-stack` is the raw stack of dynamic shadowings — the same data, but unflattened, so the debugger can show *all* bindings for a given name including those currently shadowed.

9.3 Call-stack frames

Each call-stack frame has the shape:

```
(defstruct stack-frame
  function                    ; the AutoLISP function
  form-id                     ; the form-id of the call site, or of the
                              ; currently-executing form inside this frame
  source-position             ; resolved from form-id
  bindings-introduced         ; list of (symbol . old-value-or-:unbound)
                              ; pairs, in binding order - the shadowings
                              ; this frame pushed onto the binding stack
  )
```

`bindings-introduced` lets the user navigate the call stack and see, per frame, which names that frame *introduced* (formals plus `/`-locals from §7's `bound-names`), and what those names *had been bound to* before the frame pushed its shadowings. This is the closest analog AutoLISP has to "frame-local variables": names whose current binding is owned by this specific frame.

When the user selects a frame in the UI (`cmd-select-frame`, §17.2), the REPL pane (in any UI) evaluates forms with the binding state *as it was when that frame's stopping point was reached*. For frames other than the innermost, this means temporarily unwinding the shadowings pushed by deeper frames before lookup, then restoring them. The runtime offers `clautolisp.runtime:with-frame-bindings frame &body body` to do this safely.

9.4 The binding stack and shadowed bindings

The full binding stack is a list of entries, innermost first:

```
(defstruct binding-entry
  symbol                        ; the AutoLISP name being bound
  value                         ; current value of this binding
  frame                         ; the stack-frame that introduced it, or NIL
                                ; for top-level bindings
  )
```

The user may inspect any entry directly, even one currently shadowed. In the inspector, asking for "all bindings of FOO" produces the chain of binding-entries whose `symbol` is `FOO`, from innermost to outermost; the first is what plain lookup returns, the rest are shadowed.

The debugger may **write** any binding-entry:

- Writing the innermost entry for a name is equivalent to `(setq name new-value)` at the stopping point and is the common case.
- Writing a shadowed entry changes that specific binding without affecting the innermost. When the function whose frame introduced the innermost shadow returns, the modified outer binding becomes visible again.

This second case has no syntactic equivalent in AutoLISP source code, which is exactly why the debugger should offer it: a user investigating "why is FOO 17 when my function thought it was setting it to 3" can locate the shadowing and the shadowed value without writing instrumented =setq=s.

9.5 Safety of writes

Following [Strandh2020 §4]: the compiler must not propagate type or structure information *across* poll points. Any runtime operation that depends on such information must be preceded by an explicit test, with no poll point between the test and the operation.

In AutoLISP this argument is straightforward because variable access is, semantically, symbol lookup — the compiler does not generally hold an unboxed or type-specialised version of a variable's value in a register across a poll point. The debugging body (§5.2) further guarantees this by disabling type-inference propagation across poll points. The consequence:

- Writing any binding-entry through `cmd-set-variable` is safe: the next instrumented form will pick up the new value via ordinary lookup.

- The value being written must be a valid AutoLISP value (integer, real, string, symbol, list, ename, selection set, file descriptor, VLA object, or `nil`). The runtime provides `clautolisp.runtime:coerce-from-cl` for the debugger UI to validate or convert values typed by the user.
- AutoLISP is dynamically typed, so any valid AutoLISP value is type-compatible with any binding. There is no per-variable declared type to violate.

9.6 Globals and lazy capture

Strictly speaking, in AutoLISP every symbol that isn't currently shadowed by a frame is "global." Pragmatically, the debugger distinguishes:

- **Frame-introduced names** (those listed in some frame's `bindings-introduced`) — shown in that frame's view.
- **Globals proper** — names with no current shadowing in any frame on the stack.

`globals-touched` in the snapshot is populated lazily: the instrumentation records globals *referenced or set* in the application thread since the session began, so the inspector can offer a focused view ("what globals has this run actually used"). The full global namespace is accessible through `clautolisp.runtime:list-globals` on demand; it is not included in the default snapshot for size reasons.

10. AutoLISP error handling

AutoLISP exposes two error mechanisms [AutoLISP-DG]:

1. The `*error*` function-valued global, called by the runtime with a single string argument when an uncaught error occurs at top level.
2. `vl-catch-all-apply`, which calls a function with arguments and returns either the result or an opaque error object, queryable with `vl-catch-all-error-p` and `vl-catch-all-error-message`.

The debugger interacts with both:

10.1 Interception of `*error*`

When a debug session is active on a thread, clautolisp's runtime intercepts the `*error*` call path. Before calling the user's `*error*` function, it:

1. Builds a snapshot (the call stack is the AutoLISP stack at the error point).
2. Sends an `:unhandled-error` message to the debugger.
3. Blocks on its inbound queue.

The debugger may:

- `:continue`-with-error — invoke the user's `*error*` as it would have been called originally. (This is the default for unhandled top-level errors.)
- `:continue`-with-return — supply a return value for the form that errored. This requires care: the runtime must validate that the error happened *inside* a form whose continuation is well-defined. For form-internal errors (e.g. `(/ 1 0)`), this works. For errors raised by AutoCAD/host primitives, this is generally not safe and the debugger should not offer the option.
- `:abort` — unwind the application thread cleanly, restoring system variables (this is what user `*error*` handlers traditionally do — see [AutoLISP-DG]).

The user's `*error*` handler is shown as a *frame* in the snapshot's call-stack at the point of error, so the user can choose to step into it.

10.2 Interaction with `vl-catch-all-apply`

`vl-catch-all-apply` is implemented in clautolisp as a wrapper around CL's condition system: the called function is invoked inside a `HANDLER-CASE` that converts any signaled condition into an AutoLISP error object.

When debug mode is on, the runtime additionally:

1. Records the dynamic frame as a `vl-catch-all-apply` frame in the per-thread `catch-stack`.

2. On error inside the catch: sends a `:caught-error` message to the debugger *before* the AutoLISP-visible error object is returned to the caller. The debugger may inspect the error and choose `:continue` (let the catch return the error object normally) or set a breakpoint somewhere up the stack and continue (essentially a "break on caught error" option, off by default).

10.3 No condition system in AutoLISP

There is no AutoLISP equivalent of `HANDLER-BIND`, `HANDLER-CASE` with multiple types, restarts, or `INVOKE-DEBUGGER`. Therefore:

- The debugger does **not** offer "restart" choices to the user in the AutoLISP-facing UI. The closest analog is `:continue`-with-return (§10.1).
- The debugger implementation, written in CL, freely uses the host condition system internally. This is invisible to the AutoLISP programmer.

11. Poll points: implementation

11.1 The poll-point routine

The routine called by every instrumented `poll-point k <:before|:after>` is, in CL pseudocode:

```
(declaim (inline poll-point))
(defun poll-point (form-id when)
  (let ((ti *thread-debug-info*))           ; thread-local, fast access
    (when (and ti (ti-debug-flag ti))       ; (a) debug-flag fast path
      (let* ((fn  (ti-current-function ti))
             (h   (logxor (sxhash fn) form-id)))
        (when (logbitp (mod h 1024) (ti-summary ti))            ; (b) Bloom
          (let ((bp (find-breakpoint (ti-breakpoints ti) fn form-id)))
            (when (and bp (breakpoint-active-p bp when))        ; (c) hash
              (handle-breakpoint ti bp when)))))))
```

The hot path is required to be allocation-free. Implementations must therefore *not* construct a cons or list as a hash-table key on each call; either combine `(function, form-id)` into a `fixnum` hash inline (as above), or use a two-level table indexed first by `function`, then by `form-id`. The illustrative `find-breakpoint` above hides this choice.

In ordinary code, this routine is never called — the ordinary body does not contain `poll-point` calls at all (§5).

In debugging code with `debug-flag` clear (e.g. a thread that started in debug mode but the user has currently no breakpoints anywhere), the cost is one read of a thread-local and one branch. With the flag set, the Bloom filter eliminates almost all real work for poll points that aren't breakpoints. Following [Strandh2020 §4], this is what makes "every poll point is always there" tolerable.

11.2 The summary bit table as a Bloom filter

The summary bits are managed as in [Strandh2020 §4]:

- Setting a breakpoint: insert into the hash table; set the bit at `(mod (sxhash key) 1024)`.
- Clearing a breakpoint: remove from the hash table; *do not* immediately clear the corresponding bit (it might still cover another breakpoint with a hash collision). Instead, periodically — or whenever the breakpoint table shrinks substantially — rebuild the bits from the table.

False positives are tolerated (they cost one hash-table lookup); false negatives are not.

11.3 Interpreted-code variant

In the v1 interpreter (no compiler yet, see §13), the interpreter's `EVAL` recursively calls itself on sub-forms. The instrumentation is a wrapper:

```
(defun eval-form (form env)
  (let ((id (form-id form)))
    (when *debug-mode*
      (poll-point id :before))
    (let ((result (eval-form-no-instr form env)))
      (when *debug-mode*
        (poll-point id :after))
      result)))
```

`*debug-mode*` is `T` for the application thread, `NIL` for every other thread — including the debugger thread, which never sees instrumentation overhead. This is the interpreter analog of "the ordinary entry point of every callee."

When the compiler arrives (§13), the two-bodies discipline replaces this dynamic check with a static one.

12. Adding and removing breakpoints

The debugger sets a breakpoint by calling, in its own thread:

```
(clautolisp.debug:add-breakpoint thread function form-id kind &key condition action)
(clautolisp.debug:remove-breakpoint breakpoint)
(clautolisp.debug:list-breakpoints &optional thread) → list of breakpoint
```

`add-breakpoint` is responsible for: (a) inserting into the thread's `breakpoint-table`, (b) setting the summary bit. Both operations are protected by a mutex per thread-debug-info; the application thread's poll-point routine reads without locking (the worst case is a missed breakpoint on the very first hit, which is acceptable — the application stops at the next one).

Setting a breakpoint on a function never affects threads other than the one named in the call. This is essential: a function being traced or broken-on must remain callable at full speed by every other thread, including the debugger thread itself [Strandh2020 §5.1].

13. Compiler implementation status and migration

clautolisp is currently an interpreter. The full two-bodies discipline (§5) becomes available only when the compiler-to-CL is implemented. Until then:

- **v1 (interpreter only).** The single AST is shared between debug and non-debug execution; the dispatch is the `*debug-mode*` dynamic flag described in §11.3. Cost: every form evaluation in a debugged thread pays for the flag check, even when nothing is being debugged at that point. This is acceptable — debugged code is the slow path by definition.
- **v2 (compiler to CL).** Two CL functions per AutoLISP function as in §5.1. Dispatch is at the call site, via the entries vector. The interpreter remains available as a fallback for code compiled at lower optimization levels or for use during macroexpansion debugging.

The debug protocol (Part III) is identical in both versions. UI code (Part V) is unaware of which version is in use.

Part IV — The inspector

The inspector is a separate library from the debugger. The debugger uses it for displaying values at stopping points; it is also available to the user from any REPL, debugger-attached or not.

14. The inspector protocol

The inspector is a generic value browser: given an AutoLISP value, it produces a *page* with a header (the value's printed representation, truncated), a list of components (each labelled and printable), and zero or more navigation links. The inspector tracks the *origin* of the inspected value — the variable name or expression the user started from — and the *navigation path* from that origin to the currently-displayed value, so the user can save references and reconstruct accessor expressions on demand.

```
(clautolisp.inspect:inspect value &key into origin) → inspector-session
```

- `into` selects a UI sink: `:terminal`, `:ncurses`, or `:emacs`, or a function for embedding.
- `origin` is an optional AutoLISP form (an S-expression) describing where the value came from — typically a symbol naming a global, an expression like `(entget en)`, or a debugger workspace name like `$1` (§16.2). If the inspector is opened from the debugger's snapshot view by clicking on a variable named `FOO`, the origin is `FOO`. If opened on a transient value (a REPL result, the return of a traced call, a value pulled out of mid-computation), `origin` defaults to a fresh workspace name allocated by the debugger.

An inspector-session exposes:

```
session-current              → currently displayed value
session-history              → stack of (value, page, accessor) triples,
                               outermost first
session-page                 → the current page object
session-origin               → the S-expression naming the starting point
session-down idx             → descend into component idx; the component's
                               accessor (§15) is appended to the path
session-up                   → pop history
session-eval form            → evaluate form with the current value bound
```

```
                                to debugger-private name * (also bound to
                                $0 as a numbered alias)
session-path-expression       → an S-expression that, when evaluated in the
                                same binding state as the session was opened
                                in, reproduces session-current; see §15.2
session-bind target           → bind the currently-inspected value to a name;
                                see §16.1 and §16.2
```

`session-history` carries one accessor S-expression per descent step, so that `session-path-expression` can compose them into a single form. Going *up* pops the most recent step.

15. Page generation

`inspect-page-for value` is a generic function:

```
(defgeneric inspect-page-for (value)
  (:documentation "Returns an inspect-page object describing VALUE.
Each component of the returned page must carry an accessor - an
S-expression with a free variable _ that, when _ is replaced by the
parent's path expression, yields a form evaluating to the
component's value."))
```

15.1 Per-type pages and accessors

Methods are provided for every AutoLISP value type. Each component below is annotated `[accessor]` showing the S-expression template; the placeholder `_` stands for the path expression of the parent value (the component's source).

- **Numbers** (`integer`, `real`): one-line page, no components.
- **Strings**: page shows length and content; the n-th character has accessor `(substr _ n 1)`. Length is `(strlen _)` — a read-only derived property, not a descendable component.
- **Symbols**: components are `value [(eval _)]`, `function [(function _)]`, `plist [(vlax-ldata-list _)]` if applicable.
- **Cons / list**: components are `car [(car _)]`, `cdr [(cdr _)]`; extended view shows the list as indexed elements, with element n using accessor `(nth n _)`.

- **Entity name (ename)**: the inspector first computes `(entget _)` and shows DXF group codes. Group code g has accessor `(cdr (assoc g (entget _)))`. The full DXF list is reachable as the component "entget" with accessor `(entget _)`.
- **Selection set**: contained entity n has accessor `(ssname _ n)`; length is `(sslength _)`.
- **VLA object** (when ActiveX is loaded): property `prop` has accessor `(vla-get-prop _)`; methods are not descendable. Naming follows the AutoLISP `vla-get-*` convention.
- **File descriptor**: filename, position, direction — read-only derived properties without accessors (they cannot be reconstructed from the file descriptor in user code).
- **Hash table** (clautolisp extension if present): entry under key `k` has accessor `(gethash 'k _)` if `k` is a symbol, `(gethash <printed-k> _)` if a literal of another type.

Each component carries: a label, a one-line preview, the accessor S-expression, and (for descendable components) the value itself.

When `inspect-page-for` is called on a value whose accessor cannot be expressed in pure AutoLISP — for example, the result of a side-effecting computation, or an internal slot of a CL-implementation object that has no AutoLISP-visible accessor — the component is marked with accessor `:opaque`. Descending is still allowed, but `session-path-expression` (§15.2) signals that the path from the origin to this value cannot be fully reconstructed and offers a workspace binding (§16.2) as the alternative.

15.2 Composing the path expression

`session-path-expression` is computed by folding the accessor chain in `session-history` from outermost to innermost, substituting each step's accessor's `_` with the accumulated expression. Starting from `session-origin`:

- Initial expression is `session-origin`.
- For each history step with accessor `A`, the expression becomes `(SUBST origin-expr '_ A)`.

Examples:

- Origin `FOO`, descended into `car`, then into `cdr`: path expression is `(cdr (car FOO))`.
- Origin `EN` (an ename), descended into the ename's DXF view, then into group code 10: path expression is `(cdr (assoc 10 (entget EN)))`.
- Origin `$3` (a workspace-bound value, see §16.2), descended into the cons's `cdr`: path expression is `(cdr $3)`.

When any step's accessor is `:opaque`, `session-path-expression` returns two values: the *partial* expression up to the last fully-expressible point, and a keyword `:partial`. The UI presents this honestly, e.g. "path partial: `(cdr (entget EN))` then opaque descent into internal-slot".

The expression is intended to be human-readable and re-evaluatable; it is not normalised or simplified beyond the substitution. A user who wants `(cadr X)` instead of `(car (cdr X))` can edit the resulting form in the REPL.

16. Inspector as debugger sub-tool

When the debugger displays a stopping-point's bindings (§9), each binding-entry is rendered as a one-line preview plus a "descend" affordance. Descending opens an inspector session on the binding's value, with `origin` set to the AutoLISP symbol being bound. The session shares the debugger UI's display surface; closing it returns to the stopping-point view.

16.1 Binding the inspected value to a name

`session-bind target` (§14) takes the currently-displayed value of the inspector and stores it where the user can refer to it later. `target` selects one of four destinations:

- `:workspace` (default). The value is bound in the debugger's numbered workspace (§16.2). The session reports the assigned slot, e.g. `$4`. The original AutoLISP program is untouched. This is the right choice for "let me hold on to this for the next few minutes."
- `:global SYMBOL`. The value becomes the current global binding of `SYMBOL` via `(setq SYMBOL value)` in the AutoLISP global namespace. This *does* change program-visible state — subsequent AutoLISP code that references `SYMBOL` will see the new value, including code that runs after the debug session ends. Recommended for permanent fixes, not for casual inspection.

- `:frame FRAME SYMBOL`. The value is written as the innermost binding of `SYMBOL` in the binding stack of `FRAME` (one of the call-stack frames in the snapshot). If `SYMBOL` is in `FRAME`'s `bindings-introduced` (§9.3), the existing binding-entry is overwritten. If not, a new binding-entry is *not* created — the operation reports an error, because creating a frame-local binding mid-execution would violate the dynamic-extent discipline of `/`-locals (§9.1). The user can `setq` instead, which creates a global, or use `:workspace` to keep the reference outside AutoLISP entirely.

- `:setq SYMBOL`. Shorthand: equivalent to `:global SYMBOL` if no frame on the stack currently shadows `SYMBOL`, or `:frame INNERMOST SYMBOL` (where `INNERMOST` is the innermost frame on the binding stack that has a binding-entry for `SYMBOL`) otherwise. This is the semantic of evaluating `(setq SYMBOL value)` at the stopping point and is the form the UI offers by default when the user types a name without specifying a destination.

After a successful bind, the inspector's `origin` for the current session is *not* changed — the session continues from where it was. But the bound name becomes a valid origin for any future inspector session.

16.2 The debugger workspace

The debugger maintains a per-session workspace: a list of slots `$1`, `$2`, `$3`, …, plus the alias `$0` for the inspector's currently-displayed value. Slots are AutoLISP-symbol-like names but live in a debugger-private namespace that does *not* intrude on AutoLISP's global symbol table. They are bound by `session-bind :workspace` and by REPL convenience commands (§17–§20); they may be referenced from any debugger REPL, any inspector session opened thereafter, and any breakpoint condition or trace action.

Workspace slots persist across stopping points within a session and across attach/detach cycles. They are discarded when the session ends (§24).

Workspace references in expressions are resolved by the debugger's eval layer, *not* by AutoLISP's evaluator. From the AutoLISP program's point of view, `$1` is just a symbol with no binding. Only debugger-side evaluation (REPL pane, inspector `session-eval`, breakpoint conditions, trace actions) recognises the workspace names. This isolation is what makes workspace bindings safe to use even when AutoLISP code is mid-execution: there is no possibility of an AutoLISP reference seeing or modifying the workspace.

16.3 Saving the path expression

`session-path-expression` (§15.2) returns the S-expression naming the currently-displayed value from the session's origin. The UI exposes this as a copy/yank action: in the dumb-terminal UI, a command prints the expression and offers it for clipboard copy; in ncurses, a key copies to the system clipboard via OSC 52 or the equivalent; in Emacs, the expression is pushed onto the kill ring.

A common workflow:

1. User opens inspector on `EN` (an ename).
2. Descends into the DXF view, then into group code 10 (the start point).
3. Invokes "copy path" — gets `(cdr (assoc 10 (entget EN)))`.
4. Pastes it into the REPL, edits to `(setq startpt (cdr (assoc 10 (entget EN))))` for reuse.

Alternatively, the user invokes `session-bind :workspace`, gets back `$4`, and references `$4` in subsequent debugger REPL evaluation — convenient when the path expression is long or contains non-reproducible accessors (`:opaque` steps).

16.4 Read-only by default, write affordances enumerated

The inspector session is *read-only by default* with respect to the inspected object graph. Modification affordances are limited to the following, all initiated by explicit user action:

- **Binding the displayed value to a name** via `session-bind` (§16.1) — this does not modify the inspected value itself, only the binding state.
- **Rewriting a binding-entry visible on the binding stack** (§9.5) when the inspector is displaying a binding from the snapshot.
- **Evaluating arbitrary AutoLISP forms** via `session-eval`; any side effects (`entmod`, `setq`, file I/O) are the user's responsibility.

The inspector does not offer in-place mutation of nested data (e.g. clicking a value inside a DXF list to change it). For entities, the AutoLISP-correct way is `entmod`, which the user invokes from the REPL — the inspector's

`session-path-expression` and `session-bind` features make this convenient by giving the user the right expression or a workspace handle to work with.

Part V — The UI protocol

The debugger is UI-agnostic. UIs are interchangeable: a session may even switch UIs mid-run (terminate the old UI, attach a new one to the same debugger-state, resume).

17. The `debugger.ui` protocol

A UI is a CLOS class implementing a fixed set of generic functions. The debugger calls these to report state; the UI calls a fixed set of debugger entry points to drive the session.

17.1 Debugger → UI (notifications)

```
(ui-attached ui session)
(ui-detached ui)
(ui-thread-hit ui snapshot breakpoint)
(ui-thread-unhandled-error ui snapshot error-message)
(ui-thread-caught-error ui snapshot error-object)
(ui-thread-resumed ui thread)
(ui-thread-exited ui thread value-or-error)
(ui-breakpoint-added ui breakpoint)
(ui-breakpoint-removed ui breakpoint)
(ui-show-source ui source-position)
(ui-show-message ui level format-string &rest args)   ; :info | :warning | :error
```

UIs need not handle every notification; the default method is `NIL`. Notifications are issued from the debugger thread.

17.2 UI → debugger (commands)

```
(cmd-continue session &optional thread)
(cmd-step session kind &optional thread)              ; kind  {:in :out :over :call
(cmd-advance session thread function form-id)
(cmd-set-breakpoint session function form-id &key condition action steady)
(cmd-remove-breakpoint session breakpoint)
(cmd-list-breakpoints session &optional thread)
```

```
(cmd-select-frame session frame-index)
(cmd-eval session form &key in-frame)
(cmd-set-variable session binding-entry new-value)    ; see §9.4
(cmd-inspect session value &key origin)               ; see §14
(cmd-inspector-descend session component-idx)         ; see §14
(cmd-inspector-up session)
(cmd-inspector-bind session target)                   ; see §16.1
(cmd-inspector-path-expression session)               ; see §15.2 -
returns
                                                      ; an S-expression or
                                                      ; (values expr :partial)
(cmd-workspace-list session)                          ; → alist of $N → value
(cmd-workspace-clear session &optional slot)
(cmd-disable-thread-debug session thread)
(cmd-attach-thread session thread)
```

These are ordinary CL calls — the UI is in-image for terminal UIs, and for the Emacs UI they are translated to RPC by a thin shim (§20).

17.3 Source presentation

`ui-show-source` is the central UI primitive for navigation: the debugger asks the UI to display a specific source position. The UI is free to do so by any means — printing lines, scrolling a window, sending an Emacs `find-file` — and to highlight the position. Default behaviour is to show the file's surrounding lines with a marker on the indicated position.

The reverse is supported: a UI may translate a *user pointer position* (cursor in Emacs, click in ncurses, line number typed at the dumb terminal) to a source position and ask the debugger which poll point — if any — is nearest. This is `(clautolisp.debug:poll-point-at source-position)` and is what implements "click in the source to set a breakpoint" [Clordane chap-breakpoints.tex].

18. The dumb-terminal UI

The dumb-terminal UI assumes only `*standard-input*` / `*standard-output*` with no cursor control. It is the lowest-common-denominator UI; it must work over a serial line, in a logfile-style `M-x shell`, and inside a CI run.

18.1 Layout

There is no layout. Output is line-by-line. Each notification produces one or more lines prefixed by a tag:

```
DBG> Hit breakpoint #4 at FOO line 12 col 7 - (+ x y)
DBG>   x = 1
DBG>   y = "two"
DBG>   <stack: FOO ← BAR ← top>
DBG>
DBG> Command (h for help):
```

At a stopping point, the UI enters a sub-REPL with its own prompt (`DBG>`). The prompt accepts:

- Single-letter commands: `c=onfirm continue, =s` step over, `i` step in, `o` step out, `f` finish, `n` next (alias of `s`), `b` list breakpoints, `B <pos>` set breakpoint, `p <name>` print variable, `e <form>` eval form, `l` list source, `t` stack trace, `q` quit, `h` help.
- Anything that looks like an `(form)` is treated as eval-in-frame.
- Numerical arguments to commands (`s 3` = step over 3 times) are honoured for the steppers.

Source is shown numbered:

```
DBG> l
   10:   (defun frobnicate (x y / z)
   11:     (setq z (* x y))
>> 12:     (+ x y))
   13:
```

The `>>` marker is the current stopping point.

18.2 Inspector in dumb terminal

The inspector descends one component at a time, showing labelled components numbered for selection. The session header displays the *origin* and the *current path*:

```
INSPECT> origin: EN  path: (entget EN)
INSPECT> #<DXF list, 7 elements>
INSPECT>   1. type:    "LINE"          [(cdr (assoc 0 (entget EN)))]
INSPECT>   2. layer:   "0"             [(cdr (assoc 8 (entget EN)))]
INSPECT>   3. start:   (0.0 0.0 0.0)   [(cdr (assoc 10 (entget EN)))]
INSPECT>   4. end:     (10.0 5.0 0.0)  [(cdr (assoc 11 (entget EN)))]
INSPECT>   5. handle:  "2A4"           [(cdr (assoc 5 (entget EN)))]
INSPECT> [d N] descend  [u] up  [q] quit  [e form] eval
INSPECT> [p] copy path  [b $] bind to $N  [b NAME] bind to NAME  [b ! NAME] setq
```

The accessor for each component is shown in brackets. The user can:

- Type `p` to print the current path expression (`session-path-expression`, §15.2) and have it copied to a system-clipboard equivalent where available; otherwise it is printed to the terminal for manual copy.
- Type `b $` to bind the current value to the next available workspace slot; the inspector reports `bound to $4`.
- Type `b NAME` to bind to a workspace slot under the chosen name. By convention, names starting with `$` are workspace-only; bare names are an error unless `! NAME` is used.
- Type `b ! NAME` to invoke `:setq NAME` (§16.1) — bind to the AutoLISP global namespace, with frame-aware semantics.

The workspace slots are accessible from the REPL command (`e form`):

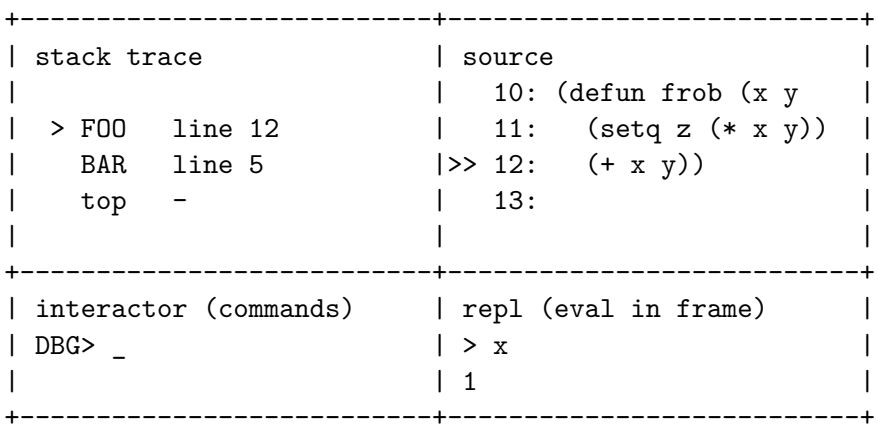
```
INSPECT> e $4
(0.0 0.0 0.0)
INSPECT> e (distance $4 $5)
12.728
```

18.3 Why this UI matters

It is the UI the implementer uses while bringing the debugger up. It is also the UI any non-Emacs user gets out of the box. Every other UI is a refinement of this; if a feature can't be expressed here, it probably shouldn't be a core debugger feature.

19. The ncurses UI (cl-charms or PDCurses)

The ncurses UI provides Strandh's four-pane layout [Clordane Documentation/chap-windows.tex] in a terminal:



19.1 Panes

- **Stack pane** (top-left): one line per frame, current frame marked. Up/Down arrows navigate frames; Enter selects.
- **Source pane** (top-right): shows the source around the current frame's stopping point. Poll points are shown as coloured markers (blue = poll point, red = breakpoint, yellow = current). Mouse click on a marker toggles the breakpoint, where mouse is supported by the terminal; otherwise, cursor movement with `b`-then-Enter does the same.
- **Interactor pane** (bottom-left): the command line, with history and completion. Same commands as §18.1, plus the `,`-prefixed command syntax from CLIM-style interactors for richer commands.
- **REPL pane** (bottom-right): a read-eval-print loop whose binding state is the current frame's view of the binding stack (§9). Variable names — including names that the selected frame introduced via its formals or `/`-locals — resolve to their values as of the selected frame's stopping point.

The layout is configurable; the user can collapse the stack pane or swap interactor and REPL.

19.2 Inspector in ncurses

Opening the inspector replaces the source pane with an inspector pane (push to stack of panes). `Esc` pops back. The component list is navigable with arrows; Enter descends, Backspace ascends.

The pane displays a *header line* showing the session origin and current path, and a *footer line* listing the available actions:

```
inspect: EN  →  (cdr (assoc 10 (entget EN)))

  car    0.0
> cadr   0.0
  caddr  0.0

[Enter] descend  [BS] up  [p] copy path  [b] bind  [e] eval  [q] quit
```

Key bindings in the inspector pane:

- `Enter` — descend into the highlighted component
- `Backspace` — pop the navigation history
- `p` — copy `session-path-expression` to the system clipboard (via OSC 52 where the terminal supports it; otherwise to a fallback debugger-side buffer also accessible as `$path` in the REPL)
- `b` — bring up a small binding dialog: a one-line input with completion offering "next workspace slot ($N)" by default, or accepting any name; prefixing the name with `!` selects the `:setq` destination (§16.1)
- `e` — switch focus to the REPL pane with the current inspector value pre-bound to `$0`
- `q` — close the inspector and return to the previous pane

19.3 Implementation notes

- Either `cl-charms` (Common Lisp curses binding) or PDCurses (via CFFI) is acceptable; the UI is written against a thin abstraction layer `clautolisp.ui.tui` that wraps both. Choice is platform-dependent: `cl-charms` on Unix, PDCurses on Windows.
- Colour fallback: 8-colour mode is sufficient. Mouse support is optional; key-only navigation must be complete.

- The ncurses event loop runs in the UI thread. Notifications from the debugger thread are funnelled through a queue and dispatched on the next refresh; no curses call from any thread other than the UI thread.

20. The Emacs UI

The Emacs UI exposes the debugger over a small RPC to an Emacs minor mode `clautolisp-debug-mode`, modeled on SLDB [SLIME]. The aim is parity with SLDB's interactive feel, not its protocol.

20.1 RPC

A simple line-oriented S-expression RPC over a socket. Messages are CL forms; the Emacs side reads them with `read` from Elisp. Format:

```
(<message-tag> <args>...)
```

Examples:

```
(:hit 4 "FOO" #s(source-position ...) ((x . 1) (y . "two")) ...)
(:continue)
(:set-breakpoint "FOO" 12 nil nil :steady t)
```

No length prefixes, no encoding negotiation, no Bencode. The transport is whatever the user's existing CL Emacs setup uses; SLIME's `swank` socket may be reused if the user is already running SLIME (clautolisp's REPL can be itself a swank-compatible REPL, but this is out of scope for this spec).

20.2 Buffers

`clautolisp-debug-mode` provides:

- **`*clautolisp-debug*` buffer**: top-level command interface. Shows current stopping point summary, key bindings as a footer. Modelled on SLDB.
- **Source buffer overlays**: open files have overlay markers at every poll point — face `clautolisp-poll-point-face`. Breakpoints get the more emphatic `clautolisp-breakpoint-face`. Current stopping point gets a transient highlight on the form.

- **`*clautolisp-bindings*`**: the current frame's view of the binding stack (§9). Each binding-entry is a line: name, current value preview, and (where applicable) an indicator that the binding is shadowing an outer entry. `RET` on a line opens an inspector buffer on that value, with origin set to the binding's symbol. `M-RET` opens a small menu offering `setq` rewrite of the binding.
- **`*clautolisp-stack*`**: frames; `RET` selects.
- **`*clautolisp-inspect*`**: inspector pages. Header line shows origin and path. `RET` on a component descends; `q` pops history; the keys below are mode-local.
- **`*clautolisp-workspace*`**: a list view of the debugger workspace (§16.2). Each slot `$N` is a line with value preview; `RET` opens the inspector on that slot, `d` removes the slot, `C-x C-s` saves the slot's path expression to the kill ring.

Inspector-buffer key bindings:

```
RET       descend into component at point
TAB       next component
S-TAB     previous component
DEL/BS    pop one step (back to parent)
p         copy path expression to kill ring
b         bind to workspace slot ($N), prompted (default: next free)
B         bind via :setq SYMBOL - prompts for the AutoLISP name
M-RET     copy current value to *clautolisp-debug* REPL as a fresh $N
e         eval - opens minibuffer, value bound to $0 and *
v         visit the value's source-of-truth (for enames: select in the
          drawing; for symbols defined in a file: open the file at defun)
q         pop inspector buffer
```

The `M-RET` binding follows SLIME's `slime-trace-dialog-copy-down-to-repl` and SLY's "Copy to REPL" convention [SLIME]; this is the idiom Emacs users expect for "give me a name for this value."

A debugger-side variable `$path` always holds the last expression yielded by `p`, accessible from any REPL evaluation. Repeated `p` invocations overwrite it; explicit binding with `b` or `B` is the way to keep more than one path around.

20.3 Key bindings (defaults)

In `clautolisp-debug-mode`:

```
c           continue
s           step over          (sldb-style: s = step)
i           step in
o           step out
f           finish
SPC         step (last-used kind)
RET         select frame
e           eval in frame
v           show source for selected frame
b           toggle breakpoint at point
C-c C-b     set breakpoint with condition
d           inspect value at point
q           quit (detach debugger)
```

These mirror SLDB where possible to minimise cognitive load for SLIME users.

20.4 What the Emacs UI does *not* do

It is not a full editor integration. It does not handle:

- Loading or compiling clautolisp source — that is the clautolisp-mode major mode's concern (separate spec).
- Completion or argument hints — those belong to `eldoc` and an inferior-lisp-style backend.
- Threaded view — only the active debugged thread is shown; switching threads is via a command (`C-c t`), not a permanent UI element.

This keeps the UI's scope coherent with the other two UIs. A feature missing in dumb terminal is missing in Emacs.

Part VI — Sessions and lifecycle

21. Starting a session

```
(clautolisp.debug:start-session :ui :terminal | :ncurses | :emacs
```

```
                                  :on-error t       ; break on unhandled error
                                  :on-warn  nil)
  → session
```

`start-session` creates a debugger object, attaches a UI, and returns the session. The session is bound to the calling thread; the calling thread becomes the *debugger thread*. Subsequent `(clautolisp.debug:debug-call thunk)` calls run `thunk` in a new application thread that is registered with this session.

For interactive use, the typical entry point is wrapping AutoLISP execution:

```
(clautolisp.debug:with-session (:ui :ncurses)
  (clautolisp:load-file "drawing.lsp")
  (clautolisp:invoke 'c:my-command))
```

22. Attaching to a running thread

A thread that is already running AutoLISP code may be attached:

```
(clautolisp.debug:attach-thread session thread)
```

The thread must be running *debug-compatible* code — either the interpreter (which always honours `*debug-mode*`) or compiled code with debugging entry points available. The `attach-thread` call sets the thread's debug flag at the next poll point. If the thread is currently executing the ordinary body of a compiled function, attachment will not take effect until that function returns to a debugging frame (in practice: returns to a caller that itself is being debugged, or returns to the top level). This is a limitation inherent to the two-bodies design [Strandh2020 §6] and is the price of zero-overhead production code.

23. Multiple debugged threads

The protocol supports multiple application threads per session. Each has its own `thread-debug-info`. Breakpoints are *per thread*; setting "break at FOO line 12" actually means "break at FOO line 12 *in this thread*." A breakpoint that should fire on every thread is set by iterating over `session-threads`.

This per-thread scoping is what makes it safe to break on system functions: the debugger thread itself is running the same code (e.g. the printer), and a global breakpoint would deadlock the debugger.

24. Session termination

The session ends when:

- The user issues `q` (or the UI calls `cmd-quit`).
- The last application thread exits and `:auto-quit-on-empty` is `T`.
- The CL host signals an uncaught condition in the debugger thread (a bug in the debugger itself).

On termination, the session detaches the UI, clears all breakpoints, restores the original `*error*` paths in every registered thread, resets thread debug flags, and discards the debugger workspace (§16.2). Application threads that are still running continue without instrumentation overhead at their next entry into ordinary code. Any bindings the user made via `:global` or `:setq` (§16.1) persist — they were written into the AutoLISP global namespace and are the user's responsibility from then on.

Part VII — Compliance and conformance

25. What a clautolisp implementation MUST provide

Phrased per [RFC2119]:

1. The reader **MUST** attach source positions to every cons cell and atom produced from a source file.
2. The runtime **MUST** expose the `clautolisp.debug` package as specified in §8.
3. The interpreter **MUST** honour `*debug-mode*` as described in §11.3.
4. The runtime **MUST** intercept the `*error*` path when debugging is active on a thread (§10.1).
5. The runtime **MUST** record `vl-catch-all-apply` frames in the per-thread `catch-stack` when debugging is active (§10.2).
6. The compiler, if present, **MUST** implement the two-bodies discipline (§5) and **MUST** emit debug metadata records (§7).

26. What a UI MUST provide

1. A UI **MUST** implement `ui-attached`, `ui-detached`, `ui-thread-hit`, `ui-thread-unhandled-error`, and `ui-thread-resumed` (§17.1).
2. A UI **MUST** provide a way for the user to issue `cmd-continue`, all five `cmd-step` kinds, `cmd-set-breakpoint`, `cmd-remove-breakpoint`, and `cmd-eval` (§17.2).
3. A UI **SHOULD** provide `cmd-inspect`, `cmd-select-frame`, `cmd-set-variable`, `cmd-inspector-bind`, and `cmd-inspector-path-expression` (§17.2). Their absence reduces but does not break the UI; in particular, a UI without `cmd-inspector-bind` cannot offer the workspace feature (§16.2) interactively, but the workspace remains accessible through `cmd-eval`.
4. A UI that implements `cmd-inspect` **MUST** display the session origin and path expression (§14, §15.2) somewhere visible while the inspector is active, so the user can see what S-expression names the currently-displayed value.
5. A UI **MUST NOT** make blocking CL calls into the debugger from arbitrary threads. UI commands run in the UI thread; the debugger thread is reached through queue messages (or, for in-image UIs, through synchronous calls that the debugger guarantees are fast and lock-free).

27. Versioning

The protocol is versioned by `clautolisp.debug:*protocol-version*`. A UI checks this at `ui-attached` and refuses to attach if its expected version differs in the major component. Minor-version mismatch is tolerated; the UI ignores notifications it does not understand.

Part VIII — Open questions and deferred work

These are flagged for v2 or later:

- **Inlining across function boundaries.** Currently disallowed in the debugging body (§5.2). It may turn out feasible to propagate debug metadata through inlining, as Strandh notes [Strandh2020 §5.2]. This

would enable breakpoints in `car`, `+`, etc., at the cost of significant compiler complexity.

- **Instruction-level stepping.** Not meaningful for the interpreter. For compiled code, instruction-level stepping into the host CL implementation's machine code is out of scope; we are not Allegro's debugger [Strandh2020 §3.4].
- **Persistent breakpoints across sessions.** The breakpoint table is in-memory and lost when the host CL exits. A future serialisation to a per-project `.clautolisp-breakpoints` file is desirable.
- **Remote debugging.** Same-image debugger and application thread is the design [Strandh2020 §4]. The Emacs UI's RPC is the only network-capable surface. Cross-machine debugger application is explicitly not pursued.
- **Reverse debugging / record-and-replay.** Out of scope.

References

```
@inproceedings{Strandh2020,
  author    = {Robert Strandh},
  title     = {Omnipresent and low-overhead application debugging},
  booktitle = {Proceedings of the 13th European Lisp Symposium (ELS 2020)},
  year      = {2020},
  pages     = {8--15},
  address   = {Z\"urich, Switzerland},
  publisher = {European Lisp Symposium},
  note      = {Slides at \url{https://european-lisp-symposium.org/static/2020/strandh
}

@misc{Clordane,
  author       = {Robert Strandh},
  title        = {Clordane: A Debugger for {Common Lisp} systems},
  howpublished = {\url{https://github.com/robert-strandh/Clordane}},
  year         = {2016--2024},
  note         = {Prototype implementation and design manual; the manual chapters \te
}

@misc{SICL,
```

```
  author       = {Robert Strandh},
  title        = {{SICL}: A fresh implementation of {Common Lisp}},
  howpublished = {\url{https://github.com/robert-strandh/SICL}},
  year         = {2008--2024},
  note         = {See in particular the \texttt{Papers/Debugging/} directory.}
}

@manual{AutoLISP-DG,
  organization = {Autodesk},
  title        = {{AutoLISP} Developer's Guide},
  year         = {2024},
  note         = {Documents the \texttt{*error*} function-valued global and the \text
}

@misc{SLIME,
  author       = {SLIME Contributors},
  title        = {{SLIME}: The Superior Lisp Interaction Mode for Emacs},
  howpublished = {\url{https://github.com/slime/slime}},
  note         = {\texttt{SLDB} is SLIME's debugger UI, the model for our Emacs UI's
}

@techreport{RFC2119,
  author      = {Scott Bradner},
  title       = {Key words for use in {RFC}s to Indicate Requirement Levels},
  institution = {Internet Engineering Task Force},
  year        = {1997},
  type        = {RFC},
  number      = {2119},
  url         = {https://www.rfc-editor.org/rfc/rfc2119}
}
```

Additional credits, per [Strandh2020]:

- *Frode Fjeld* — the use of multiple entry points per function to distinguish ordinary from debugging code paths.
- *Michael Raskin* — the discipline of emitting two function bodies in every compilation, so the choice between debug-friendly and optimised code is made by the call site, not by the compiler invocation.

Both ideas are essential to the design in Part II and are adopted here without modification.